CAIRO UNIVERSITY
Faculty of Computing and AI

ASSIGNMENT 2

Vole Machine

Assignment Report

COURSE
OOP
CS213

COURSE INFO	Supervision	SECTION	DATE
CS213 (OOP)	Mohammad El-Ramly	S5-S6 & S23-S24	27 / 10 / 2026

TEAM LEADER	TEAM LEADER PHONE
Abdelrahman Ahmed Elsayed	01110734152

TEAM MEMBERS

#	FULL NAME	STUDENT ID	EMAIL
01	Abdelrahman Ahmed ELsayed	20250346 , S5	20250346@stud.fci-cu.edu.eg
02	Mohamed Hossam Abdelsamad	20250554, S5	20250554@stud.fci-cu.edu.eg
03	Mohamed Ramy Mohamed	20250560, S5	20250560@stud.fci-cu.edu.eg
04	Mahmoud Ehab Ali	20250606 ,S23	20250606@stud.fci-cu.edu.eg

WORK BREAKDOWN (WHO DID WHAT)

MEMBER	WHO DID WHAT
Abdelrahman Ahmed	*Architecture of the vole machine , Initial design of the machine , AI final design (Gemini) , Storage , Memory , Register*
Mohamed Hossam	*AI Design (DeepSeek) , Machine*
Mohamed Ramy	*The 2 Vole machine programs , The 2 Videos , AI Design (Chat GPT) , CPU*
Mahmoud Ehab	*AI Design (Claude) , ALU*

SECTION 01

INTRODUCTION

01

THE VOLE MACHINE

The Vole Machine is a simplified computer simulator designed to demonstrate how a computer executes machine instructions. It consists of a CPU, memory, registers, and an Arithmetic Logic Unit (ALU). The CPU follows the fetch–decode–execute cycle to retrieve instructions from memory, interpret them, and perform operations such as arithmetic, logical operations, and data transfer. By implementing the Vole Machine in C++ using OOP, we can understand basic computer architecture and apply concepts such as encapsulation, abstraction, inheritance, and composition.

ARCHITECTURE

CPU & CONTROL UNIT

The CPU (Central Processing Unit) executes instructions in the Vole Machine. The Control Unit manages the execution process, while the Program Counter (PC) stores the address of the next instruction and the Instruction Register (IR) holds the current instruction being executed. Together, they enable the CPU to follow the fetch–decode–execute cycle.

REGISTERS

The Vole Machine contains 16 general-purpose registers, named R0 to R15, each with a size of 2 hexadecimal digits (8 bits). Registers provide fast temporary storage for data and intermediate results during instruction execution, allowing the CPU to perform operations efficiently without constantly accessing memory.

MAIN MEMORY

The Vole Machine has 256 memory cells, addressed from 00 to FF in hexadecimal. Each cell stores 8 bits (2 hexadecimal digits), and each address identifies a single cell. Memory stores program instructions and data that the CPU reads from or writes to during execution.

INSTRUCTION SET

Each Vole Machine instruction consists of 4 hexadecimal digits (16 bits), divided into an opcode and operands that specify the operation and the required registers or memory addresses. The operations include data transfer, arithmetic calculations, logical operations, conditional branching, and input/output, allowing the CPU to execute programs step by step.

SECTION 02

INITIAL DESIGN

02

CLASSES & RESPONSIBILITIES

CLASS	DETAILS (attributes & methods)
Machine	*Attributes: cpu, memory, screenBuffer* *Methods: Machine, displayMenu, loadProgramFile, overwriteValue, runAtOnce, runStepByStep, displayState, resetMachine*
CPU	*Attributes: pc, ir, halted, reg, alu* *Methods: CPU, fetch, isValidInstruction, decodeAndExecute, runNextStep, setPC, getPC, getIR, isHalted, getRegisters, getALU, reset*
Register	*Attributes: regs* *Methods: Register, getReg, setReg, clear*
ALU	*Attributes: (None)* *Methods: hexToDec, decToHex, isValidHex, addTwosComplement, addFloat, bitwiseOr, bitwiseAnd, bitwiseXor, rotateRight*
Memory	*Attributes: cells* *Methods: Memory, getCell, setCell, clear*

INITIAL
DESIGN

PLANTUML & UML
Our initial design

PLANTUML CODE

```
@startuml SimplifiedVoleMachine

skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam class {
    BackgroundColor #F8FAFC
    BorderColor #334155
    ArrowColor #0F172A
    HeaderBackgroundColor #E2E8F0
}

class Machine {
    - cpu : CPU
    - memory : Memory
    - screenBuffer : string
    + Machine()
    + displayMenu() : void
    + loadProgramFile(filename : string, startAddr : int) : bool
    + overwriteValue() : void
    + runAtOnce() : void
    + runStepByStep() : void
    + displayState() : void
    + resetMachine() : void
}

class Memory {
    - cells : string[256]
    + Memory()
    + getCell(address : int) : string
    + setCell(address : int, val : string) : void
    + clear() : void
}

class CPU {
    - pc : int
    - ir : string
    - halted : bool
    - reg : Register
    - alu : ALU
    + CPU()
    + fetch(mem : const Memory&) : void
    + isValidInstruction(inst : string) : bool
    + decodeAndExecute(mem : Memory&, screenBuffer : string&) : void
    + runNextStep(mem : Memory&, screenBuffer : string&) : bool
    + setPC(addr : int) : void
    + getPC() : int
    + getIR() : string
    + isHalted() : bool
    + getRegisters() : Register&
    + getALU() : ALU&
    + reset(startAddr : int) : void
```

```
}

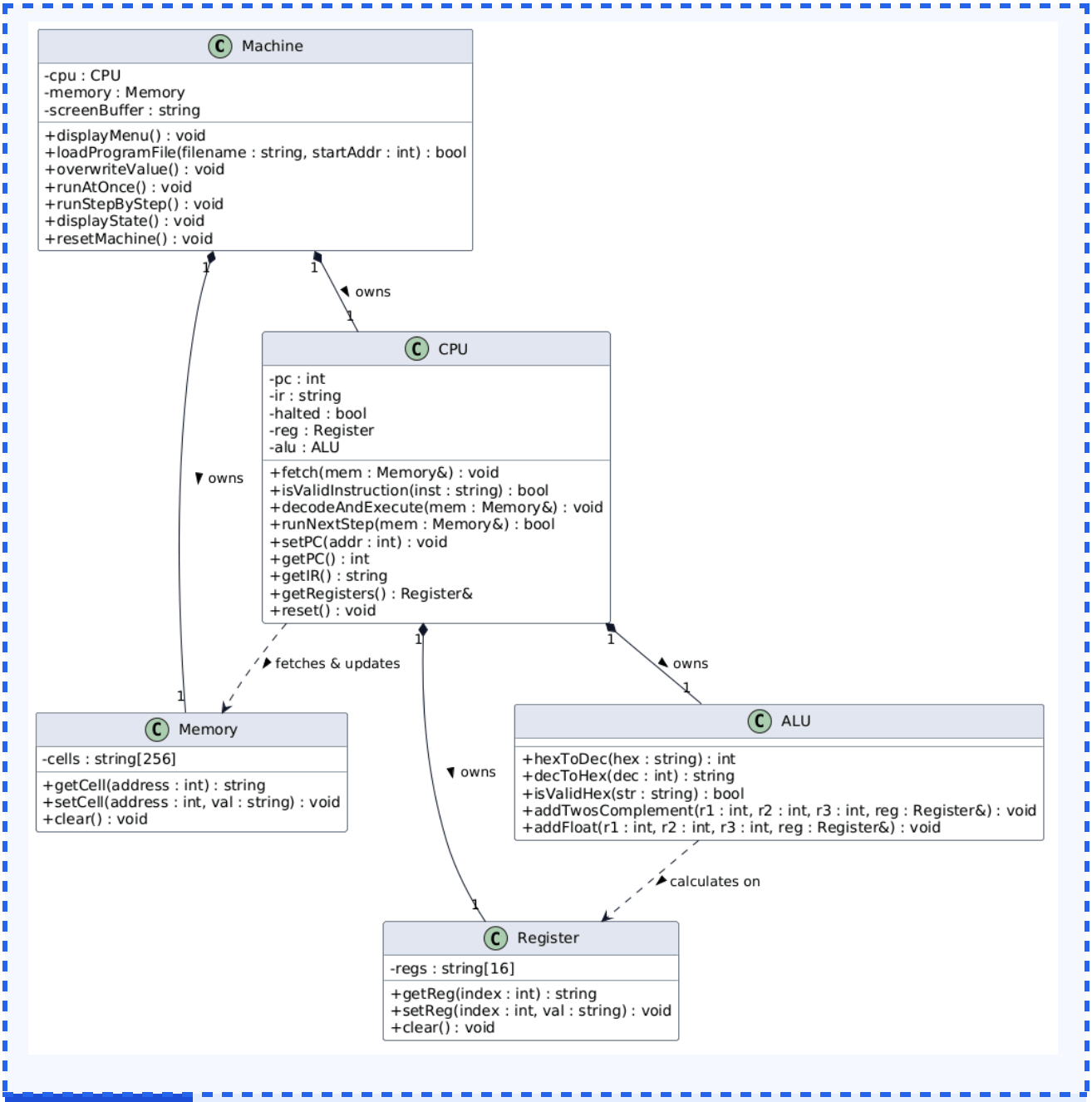
class Register {
    - regs : string[16]
    + Register()
    + getReg(index : int) : string
    + setReg(index : int, val : string) : void
    + clear() : void
}

class ALU {
    + hexToDec(hex : string) : int
    + decToHex(dec : int) : string
    + isValidHex(str : string) : bool
    + addTwosComplement(r1 : int, r2 : int, r3 : int, reg : Register&) : void
    + addFloat(r1 : int, r2 : int, r3 : int, reg : Register&) : void
    + bitwiseOr(r1 : int, r2 : int, r3 : int, reg : Register&) : void
    + bitwiseAnd(r1 : int, r2 : int, r3 : int, reg : Register&) : void
    + bitwiseXor(r1 : int, r2 : int, r3 : int, reg : Register&) : void
    + rotateRight(r : int, steps : int, reg : Register&) : void
}

' Composition (Strong Ownership)
Machine "1" *-- "1" CPU : owns >
Machine "1" *-- "1" Memory : owns >
CPU "1" *-- "1" Register : owns >
CPU "1" *-- "1" ALU : owns >

' Dependency (Usage)
CPU ..> Memory : fetches & updates >
ALU ..> Register : calculates on >

@enduml
```



INITIAL / UML UML class diagram of the initial design.

SECTION 03

AI DESIGNS

03

AI 01

Claude

CHAT LINK

https://claude.ai/share/3c0f6211-4347-4a41-95da-4448b4ccbaed

PLANTUML CODE

```
@startuml VoleMachineSimulator
skinparam classAttributeIconSize 0
hide empty members

class Constants <<namespace>> {
  +{static} MEMORY_SIZE : int = 256
  +{static} SCREEN_ADDRESS : int = 255
  +{static} REGISTER_COUNT : int = 16
  +{static} INSTRUCTION_SIZE : int = 2
  +{static} BITS_PER_REGISTER : int = 8
}

class HexUtil <<static>> {
  +{static} isHexDigit(c : char) : bool
  +{static} parseHex(text : string) : int
  +{static} toHex(value : int, width : int) : string
}

class VoleException <<exception>> {
  +what() : string
}

class ProgramRejectedException <<exception>> {
  -errors : vector<string>
  +getErrors() : vector<string>
}
VoleException <|-- ProgramRejectedException

class Screen {
  -bytes : vector<uint8_t>
  +write(value : uint8_t)
  +getBytes() : vector<uint8_t>
  +clear()
}

class Memory {
  -cells : uint8_t[256]
  -screen : Screen&
  +Memory(screen : Screen&)
  +read(address : uint8_t) : uint8_t
  +writeByCpu(address : uint8_t, value : uint8_t)
  +writeByUser(address : uint8_t, value : uint8_t)
  +readWord(address : uint8_t) : uint16_t
  +writeWord(address : uint8_t, word : uint16_t)
```

```
  +clear()
  +{static} canFetchAt(address : int) : bool
  +{static} canLoad(address : int, count : int) : bool
}
note right of Memory
  writeByCpu stores the value and, when the
  address is SCREEN_ADDRESS, also calls
  screen.write(value).
  writeByUser stores only (no screen output).
  canFetchAt: even and address + 2 <= SCREEN_ADDRESS.
  canLoad: even, address < SCREEN_ADDRESS and
  address + 2 * count <= SCREEN_ADDRESS.
  Both use int arithmetic.
end note
Memory --> "1" Screen : forwards FF writes
Memory ..> Constants

class RegisterFile {
  -regs : uint8_t[16]
  +get(index : int) : uint8_t
  +set(index : int, value : uint8_t)
  +clear()
}
RegisterFile ..> Constants
RegisterFile ..> VoleException : throws

class ALU <<static>> {
  +{static} addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
  +{static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
  +{static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
  +{static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
  +{static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
  +{static} rotateRight(value : uint8_t, times : int) : uint8_t
  -{static} decodeFloat(bits : uint8_t) : double
  -{static} encodeFloat(value : double) : uint8_t
}
note right of ALU
  Float value = (-1)^s * (m / 16) * 2^(e - 4).
  encodeFloat normalizes the mantissa to [0.5, 1)
  and truncates it to 4 bits. Overflow saturates;
  underflow and zero give 0x00.
  rotateRight uses times mod 8; 0 returns the value.
end note
ALU ..> Constants

class Instruction <<value>> {
  -word : uint16_t
  -opcode : int
  -r : int
  -s : int
  -t : int
  -xy : uint8_t
  +{static} findError(word : uint16_t) : string
  +{static} decode(word : uint16_t) : Instruction
  +getWord() : uint16_t
  +getOpcode() : int
  +getR() : int
  +getS() : int
```

```
  +getT() : int
  +getXY() : uint8_t
  +describe() : string
}
note right of Instruction
  findError returns an empty string when valid,
  otherwise the reason: unknown op-code,
  op 4 needs 0RS, op A needs R0X, op C needs 000,
  op B target must satisfy Memory.canFetchAt.
  decode throws VoleException with that reason.
end note
Instruction ..> Memory : uses canFetchAt
Instruction ..> VoleException : throws

class StepResult <<value>> {
  +address : uint8_t
  +word : uint16_t
  +description : string
  +halted : bool
}

class CPU {
  -pc : uint8_t
  -ir : uint16_t
  -halted : bool
  -registers : RegisterFile
  -memory : Memory&
  +CPU(memory : Memory&)
  +fetch()
  +step() : StepResult
  +setPC(address : uint8_t)
  +setRegister(index : int, value : uint8_t)
  +getPC() : uint8_t
  +getIR() : uint16_t
  +isHalted() : bool
  +getRegisters() : const RegisterFile&
  +resetState()
  +clearRegisters()
  -execute(instr : Instruction)
  -executeLoadFromMemory(instr : Instruction)
  -executeLoadImmediate(instr : Instruction)
  -executeStore(instr : Instruction)
  -executeMove(instr : Instruction)
  -executeAddTwosComplement(instr : Instruction)
  -executeAddFloat(instr : Instruction)
  -executeLogic(instr : Instruction)
  -executeRotate(instr : Instruction)
  -executeJump(instr : Instruction)
  -executeHalt()
}
note right of CPU
  fetch: throw if not Memory.canFetchAt(pc);
  ir = readWord(pc); pc = pc + INSTRUCTION_SIZE.
  step: fetch, Instruction.decode(ir), execute.
  executeStore calls memory.writeByCpu and has
  no special case for FF.
  executeJump: if reg R equals reg 0 then pc = XY.
  resetState clears pc, ir and halted only.
```

```
end note
CPU *-- "1" RegisterFile
CPU --> "1" Memory : reads and writes
CPU ..> Instruction : decodes
CPU ..> ALU : uses
CPU ..> StepResult : returns
CPU ..> Constants
CPU ..> VoleException : throws

class ProgramLoader {
  +load(path : string, address : uint8_t) : vector<uint16_t>
  -parseFile(path : string, errors : vector<string>&) : vector<uint16_t>
}
note right of ProgramLoader
  Never touches the machine.
  1. read the file, skip blank and comment lines,
     take 4 hex digits per line (errors carry
     the line number)
  2. Instruction.findError on every word
  3. Memory.canLoad(address, count)
  Any problem throws ProgramRejectedException
  listing every error found.
end note
ProgramLoader ..> Instruction : validates with
ProgramLoader ..> Memory : uses canLoad
ProgramLoader ..> HexUtil
ProgramLoader ..> ProgramRejectedException : throws

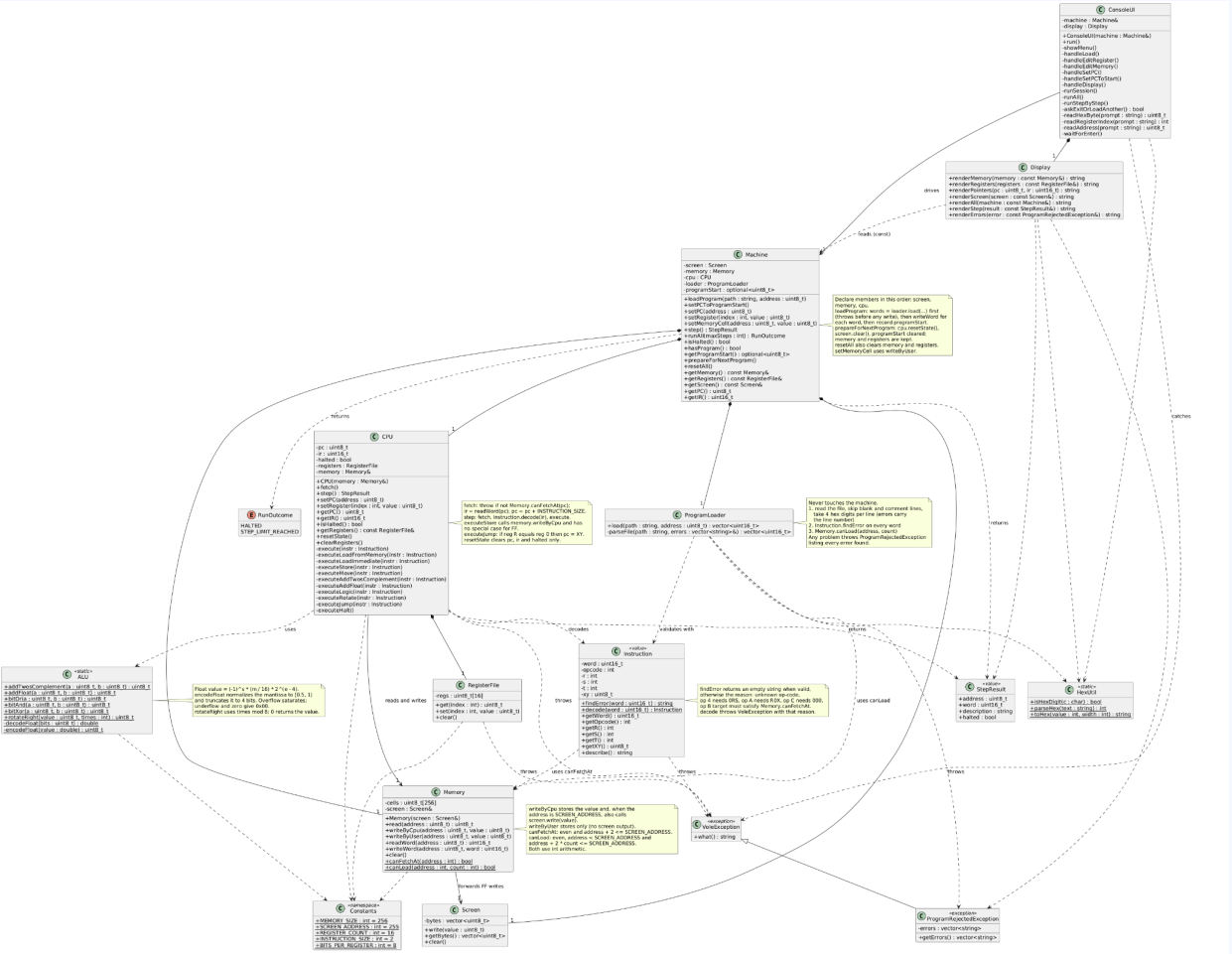
enum RunOutcome {
  HALTED
  STEP_LIMIT_REACHED
}

class Machine {
  -screen : Screen
  -memory : Memory
  -cpu : CPU
  -loader : ProgramLoader
  -programStart : optional<uint8_t>
  +loadProgram(path : string, address : uint8_t)
  +setPCToProgramStart()
  +setPC(address : uint8_t)
  +setRegister(index : int, value : uint8_t)
  +setMemoryCell(address : uint8_t, value : uint8_t)
  +step() : StepResult
  +runAll(maxSteps : int) : RunOutcome
  +isHalted() : bool
  +hasProgram() : bool
  +getProgramStart() : optional<uint8_t>
  +prepareForNextProgram()
  +resetAll()
  +getMemory() : const Memory&
  +getRegisters() : const RegisterFile&
  +getScreen() : const Screen&
  +getPC() : uint8_t
  +getIR() : uint16_t
}
note right of Machine
```

```
  Declare members in this order: screen,
  memory, cpu.
  loadProgram: words = loader.load(...) first
  (throws before any write), then writeWord for
  each word, then record programStart.
  prepareForNextProgram: cpu.resetState(),
  screen.clear(), programStart cleared;
  memory and registers are kept.
  resetAll also clears memory and registers.
  setMemoryCell uses writeByUser.
end note
Machine *-- "1" Screen
Machine *-- "1" Memory
Machine *-- "1" CPU
Machine *-- "1" ProgramLoader
Machine ..> StepResult : returns
Machine ..> RunOutcome : returns

class Display {
  +renderMemory(memory : const Memory&) : string
  +renderRegisters(registers : const RegisterFile&) : string
  +renderPointers(pc : uint8_t, ir : uint16_t) : string
  +renderScreen(screen : const Screen&) : string
  +renderAll(machine : const Machine&) : string
  +renderStep(result : const StepResult&) : string
  +renderErrors(error : const ProgramRejectedException&) : string
}
Display ..> Machine : reads (const)
Display ..> StepResult
Display ..> ProgramRejectedException
Display ..> HexUtil

class ConsoleUI {
  -machine : Machine&
  -display : Display
  +ConsoleUI(machine : Machine&)
  +run()
  -showMenu()
  -handleLoad()
  -handleEditRegister()
  -handleEditMemory()
  -handleSetPC()
  -handleSetPCToStart()
  -handleDisplay()
  -runSession()
  -runAll()
  -runStepByStep()
  -askExitOrLoadAnother() : bool
  -readHexByte(prompt : string) : uint8_t
  -readRegisterIndex(prompt : string) : int
  -readAddress(prompt : string) : uint8_t
  -waitForEnter()
}
ConsoleUI --> "1" Machine : drives
ConsoleUI *-- "1" Display
ConsoleUI ..> HexUtil
ConsoleUI ..> VoleException : catches
@enduml
```



AI 1 / UML UML class diagram generated from the PlantUML code above.

AI

02 DeepSeek

CHAT LINK https://chat.deepseek.com/share/95qbtvqg0n4mr3j772

PLANTUML CODE

```
@startuml VoleMachine_Improved
!theme plain
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName "Segoe UI"
skinparam defaultFontSize 12
skinparam class {
    BackgroundColor #F8FAFC
    BorderColor #1E293B
    ArrowColor #0F172A
    HeaderBackgroundColor #E2E8F0
    AttributeFontSize 11
    MethodFontSize 11
}

title Vole Machine Simulator — Simplified OOP Domain Model\n(5 Core Classes)

' ============================================================
'  PACKAGE: ORCHESTRATION
' ============================================================
package "Orchestration" as ORCH #DBEAFE {
    class Machine {
        - cpu : CPU
        - memory : Memory
        - screenBuffer : string
        __
        + Machine()
        + displayMenu() : void
        + loadProgramFile(filename : string, startAddr : int) : bool
        + overwriteValue() : void
        + runAtOnce() : void
        + runStepByStep() : void
        + displayState() : void
        + resetMachine() : void
    }
}

' ============================================================
'  PACKAGE: PROCESSING
' ============================================================
package "Processing" as PROC #FEF3C7 {
    class CPU {
        - pc : int
        - ir : string
        - halted : bool
        - reg : Register
        - alu : ALU
        __
```

```
        + CPU()
        + fetch(mem : const Memory&) : void
        + isValidInstruction(inst : string) : bool
        + decodeAndExecute(mem : Memory&, screenBuffer : string&) : void
        + runNextStep(mem : Memory&, screenBuffer : string&) : bool
        + setPC(addr : int) : void
        + getPC() : int
        + getIR() : string
        + isHalted() : bool
        + getRegisters() : Register&
        + getALU() : ALU&
        + reset(startAddr : int = 0) : void
    }

    class ALU {
        __
        + hexToDec(hex : string) : int
        + decToHex(dec : int) : string
        + isValidHex(str : string) : bool
        + addTwosComplement(r1 : int, r2 : int, r3 : int, reg : Register&) :
void
        + addFloat(r1 : int, r2 : int, r3 : int, reg : Register&) : void
        + bitwiseOr(r1 : int, r2 : int, r3 : int, reg : Register&) : void
        + bitwiseAnd(r1 : int, r2 : int, r3 : int, reg : Register&) : void
        + bitwiseXor(r1 : int, r2 : int, r3 : int, reg : Register&) : void
        + rotateRight(r : int, steps : int, reg : Register&) : void
    }
}

' ============================================================
'  PACKAGE: STORAGE
' ============================================================
package "Storage" as STOR #DCFCE7 {
    class Memory {
        - cells : string[256]
        __
        + Memory()
        + getCell(address : int) : string
        + setCell(address : int, val : string) : void
        + clear() : void
    }

    class Register {
        - regs : string[16]
        __
        + Register()
        + getReg(index : int) : string
        + setReg(index : int, val : string) : void
        + clear() : void
    }
}

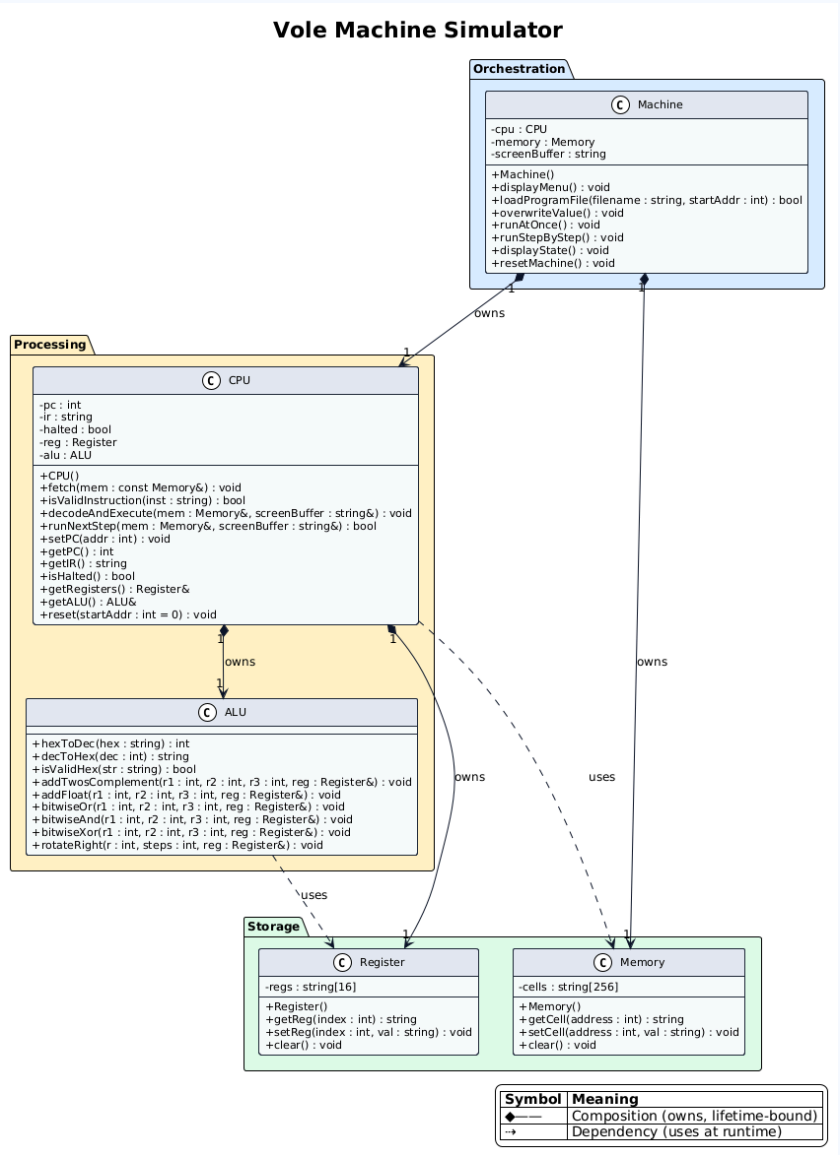
' ============================================================
'  COMPOSITION (strong ownership, solid diamond)
' ============================================================
Machine "1" *--> "1" CPU     : owns
Machine "1" *--> "1" Memory  : owns
CPU     "1" *--> "1" Register : owns
```

```
CPU     "1" *--> "1" ALU      : owns

' ============================================================
'  DEPENDENCY (uses, dashed arrow)
' ============================================================
CPU ..> Memory   : uses
ALU ..> Register : uses

' ============================================================
'  LEGEND
' ============================================================
legend right
  |= Symbol |= Meaning |
  | ◆── | Composition (owns, lifetime-bound) |
  | ⇢   | Dependency (uses at runtime) |
endlegend

@enduml
```



AI 2 / UML UML class diagram generated from the PlantUML code above.

AI

03 Chat GPT

CHAT LINK

https://chatgpt.com/share/6adf988c-3220-83ea-872e-b74d577a414b?ogimg=plain

PLANTUML CODE

```
@startuml VoleMachine

left to right direction
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam linetype ortho

class Machine {
  - cpu : CPU
  - memory : Memory
  - screenBuffer : string
  + displayMenu() : void
  + loadProgramFile() : bool
  + runAtOnce() : void
  + runStepByStep() : void
  + displayState() : void
  + resetMachine() : void
}

class CPU {
  - pc : int
  - ir : string
  - halted : bool
  + runNextStep() : bool
  + setPC(addr : int) : void
  + getPC() : int
  + getIR() : string
  + reset() : void
}

class Memory {
  - cells : string[256]
  + getCell(address : int) : string
  + setCell(address : int, val : string) : void
  + clear() : void
}

class Register {
  - regs : string[16]
  + getReg(index : int) : string
  + setReg(index : int, val : string) : void
  + clear() : void
}

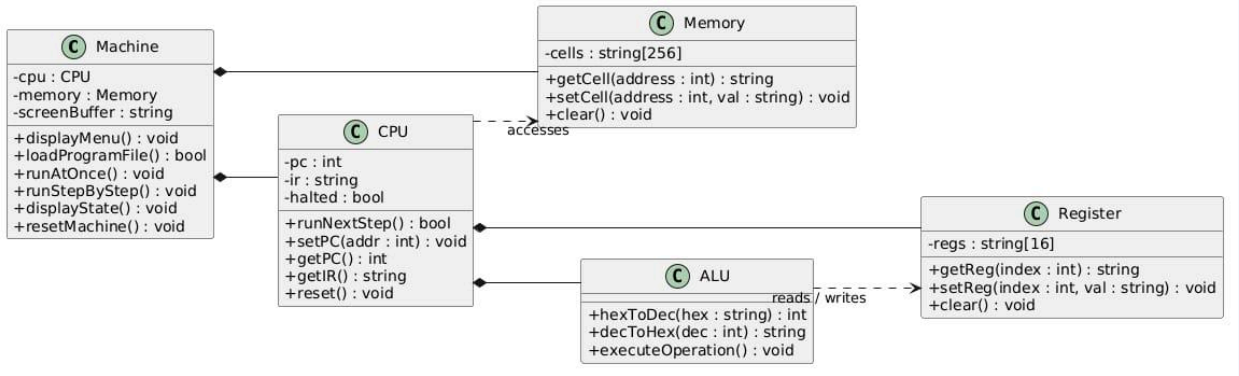
class ALU {
  + hexToDec(hex : string) : int
  + decToHex(dec : int) : string
```

```
  + executeOperation() : void
}

Machine *-- CPU
Machine *-- Memory
CPU *-- Register
CPU *-- ALU

CPU ..> Memory : accesses
ALU ..> Register : reads / writes

@enduml
```



AI 3 / UML UML class diagram generated from the PlantUML code above.

SECTION 04

FINAL DESIGN

04

FINAL

AI

FINAL DESIGN / PLANTUML & UML

GEMINI

CHAT LINK https://share.gemini.google/tc84n5fEpn8O

PLANTUML CODE

```
@startuml VoleMachine_Enhanced_Design2
!theme plain
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName "Segoe UI"
skinparam defaultFontSize 12
skinparam class {
    BackgroundColor #F8FAFC
    BorderColor #1E293B
    ArrowColor #0F172A
    HeaderBackgroundColor #E2E8F0
    AttributeFontSize 11
    MethodFontSize 11
}

title Vole Machine Simulator — Enhanced Design 2 (AIEP Compliant)\n(6 Core
Classes)

' ============================================================
'  PACKAGE: ORCHESTRATION
' ============================================================
package "Orchestration" as ORCH #DBEAFE {
    class Machine {
        - cpu : CPU
        - memory : Memory
        - screenBuffer : string

        __
        + Machine()
        + displayMenu() : void
        + loadProgramFile(filename : string, startAddr : int = 0) : bool
        + overwriteValue() : void
        + runAtOnce(maxSteps : int = 1000) : void
        + runStepByStep() : void
        + displayState() const : void
        + resetMachine() : void
    }
}

' ============================================================
'  PACKAGE: PROCESSING
' ============================================================
package "Processing" as PROC #FEF3C7 {
    class CPU {
        - pc : int
```

```
        - ir : string
        - halted : bool
        - reg : Register
        - alu : ALU

        __
        + CPU()
        + fetch(mem : const Memory&) : void
        + decodeAndExecute(mem : Memory&, screenBuffer : string&) : void
        + runNextStep(mem : Memory&, screenBuffer : string&) : bool
        + {static} isValidInstruction(inst : string) : bool
        + setPC(addr : int) : void
        + getPC() const : int
        + getIR() const : string
        + isHalted() const : bool
        + getRegisters() const : const Register&
        + setRegister(index : int, val : string) : void
        + reset(startAddr : int = 0) : void
    }

    class ALU {

        __
        + {static} hexToDec(hex : string) : int
        + {static} decToHex(dec : int, width : int = 2) : string
        + {static} isValidHex(str : string) : bool
        + addTwosComplement(val1 : string, val2 : string) const : string
        + addFloat(val1 : string, val2 : string) const : string
        + bitwiseOr(val1 : string, val2 : string) const : string
        + bitwiseAnd(val1 : string, val2 : string) const : string
        + bitwiseXor(val1 : string, val2 : string) const : string
        + rotateRight(val : string, steps : int) const : string
        - floatToDec(hexByte : string) const : double
        - decToFloat(value : double) const : string
    }
}

' ============================================================
'  PACKAGE: STORAGE (INHERITANCE & POLYMORPHISM)
' ============================================================
package "Storage" as STOR #DCFCE7 {
    abstract class Storage {
        # size : int
        # cells : vector<string>

        __
        + Storage(size : int)
        + {abstract} ~Storage()
        + virtual getCell(index : int) const : string
        + virtual setCell(index : int, val : string) : void
        + virtual clear() : void
        + getSize() const : int
        # normalizeHex(val : string) const : string
    }

    class Memory {

        __
        + Memory()
        + getCell(address : int) const : string
        + setCell(address : int, val : string) : void
        + getWord(address : int) const : string
```

```
    }

    class Register {
        __
        + Register()
        + getReg(index : int) const : string
        + setReg(index : int, val : string) : void
    }
}

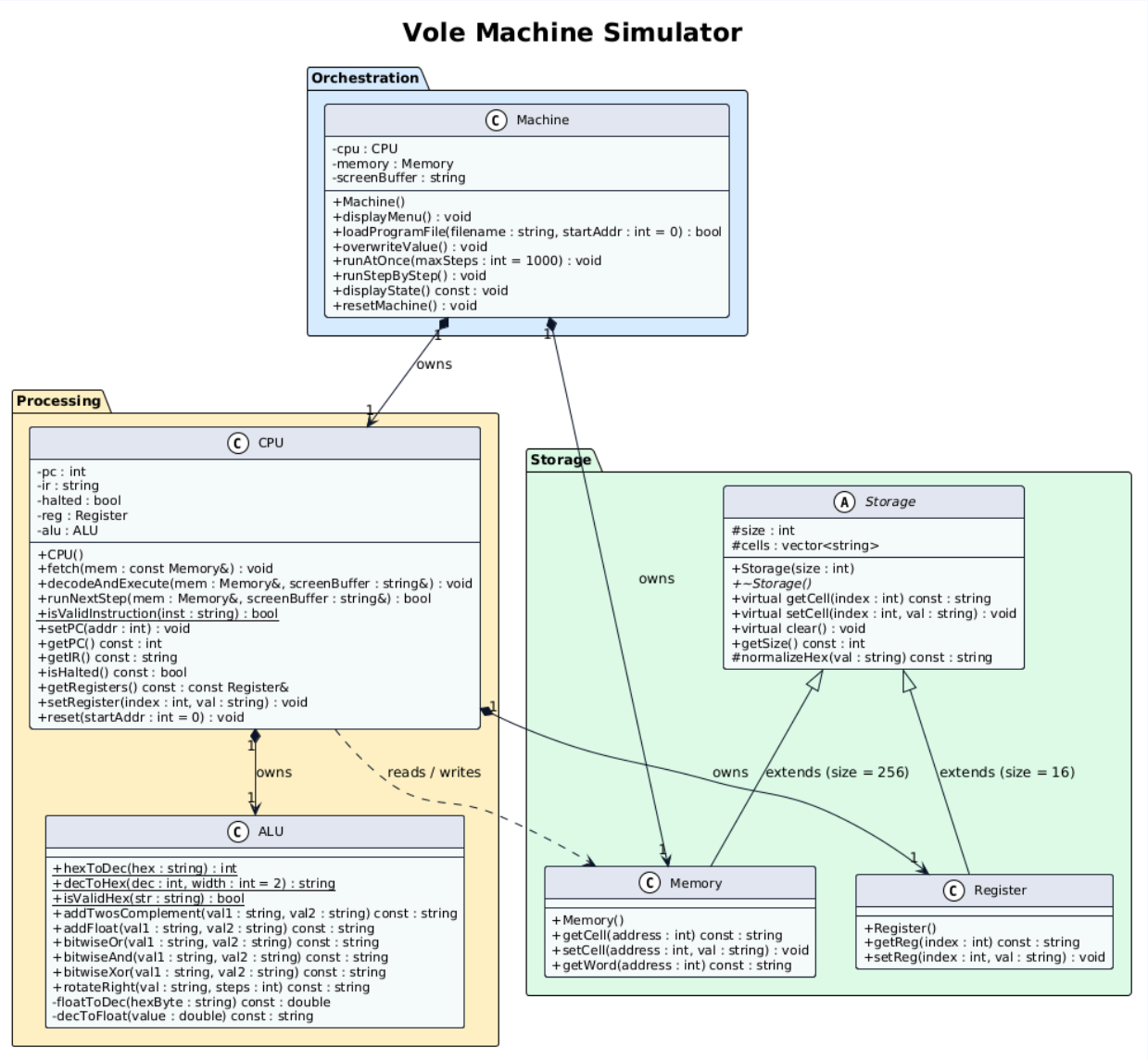
' ============================================================
' INHERITANCE (is-a, hollow triangle)
' ============================================================
Storage <|-- Memory   : extends (size = 256)
Storage <|-- Register : extends (size = 16)

' ============================================================
' COMPOSITION (strong ownership, solid diamond)
' ============================================================
Machine "1" *--> "1" CPU      : owns
Machine "1" *--> "1" Memory   : owns
CPU     "1" *--> "1" Register : owns
CPU     "1" *--> "1" ALU      : owns

' ============================================================
' DEPENDENCY (uses, dashed arrow)
' ============================================================
CPU ..> Memory : reads / writes

' ============================================================
' LEGEND
' ============================================================
legend right
  |= Symbol |= Meaning |
  | <|-- | Inheritance (is-a relationship) |
  | *--> | Composition (owns, lifetime-bound) |
  | ..>  | Dependency (uses at runtime) |
endlegend

@enduml
```



FINAL / UML UML class diagram of the final design.

FINAL

WHY

JUSTIFICATION

Why this is the best design

WHY IT IS THE BEST DESIGN

- *Full OOP (AIEP) uses Storage for inheritance and polymorphism, while keeping all data private and properly encapsulated.*
- *No duplicate code memory and Register share the same base logic instead of repeating it.*
- *Clean separation each class has one clear job—ALU only does math, CPU runs instructions, and Machine controls the flow.*
- *Easy to code and debug only 6 simple classes—not too complicated like Design 1, and not missing details like Design 3.*
- *Safe from common bugs prevents infinite loops and bad hex formatting automatically.*

KEY IMPROVEMENTS OVER THE INITIAL DESIGN

- *Added Inheritance & Polymorphism areated a Storage base class for Memory and Register so they share code and fulfill all AIEP rules.*
- *Fixed Encapsulation in CPU: Made getRegisters() read-only (const), removed getALU(), and added setRegister() to protect internal data.*
- *Separated ALU from Register removed the direct link between ALU and Register so the ALU only calculates values and returns the result to the CPU.*
- *Added Float & Hex Helpers added private float conversion methods in ALU and automatic 2-character hex formatting in Storage.*
- *Added Loop & Memory Protection added a maxSteps limit in runAtOnce() to stop infinite loops and getWord() in Memory to safely read 2-byte instructions.*

SECTION 05

EXAMPLES

05

EXAMPLE 01

VOLE PROGRAM

Vole machine program 1

WHAT IT TESTS — *Adding , Subtracting , Printing loop*

VOLE PROGRAM (machine code)

```
; ------------------------------------------------
; Vole Machine Program
; Purpose:
; 1. Load two numbers (3 and 4)
; 2. Add them -> result = 7
; 3. Subtract 2 from result (add FE, two's complement) -> result = 5
; 4. Loop: add result to a running total, 3 times -> totals 5, 10, 15
; 5. Output each step to screen
; 6. Halt
; ------------------------------------------------


2003 ;Load immediate value 03 into register 0
2104 ;Load immediate value 04 into register 1
5201 ;Add the values in register 0 and 1 , put result in register 2 (result=7)
23FE ;Load immediate value FE(that is -2) into register 3
5423 ;Add values in register 2 and 3 , put result in register 4 (result=5)
34FF ;Store register 4 into memory[FF] (screen output)


2000 ;Load immediate value 00 into register 0(need to return to the loop
beginning)
2500 ;Load immediate value 00 into register 5 (total output)
2603 ;Load immediate value 03 into register 6 (loop counter)
27FF ;Load immediate value FF(that is -1) into register 7(loop decrement)

;-----loop start (address = 14)-----

5554 ;Add values in register 5 and 4 , put result in register 5 (00+05 ,
result=5)
35FF ;Store register 5 into memory[FF] (screen output)
5667 ;Add values in register 6 and 7 , put result in register 6 (decrease
counter by 1)
B61E ;if register 6(counter)==0 ,jump to 1E (1E address of the HALT)
B014 ;if register 0 ==0 (Always true) ,jump to 14 (14 address of the loop
start)

;-----loop end-----

C000 ;HALT (address = 1E)
```

EXAMPLE

02

VOLE PROGRAM

Vole machine program 2

WHAT IT TESTS — *OR , AND , XOR , Right rotating*

VOLE PROGRAM (machine code)

```
; ------------------------------------------------
; Vole Machine Program 2
; Purpose:
; 1. Load two numbers (0F and 3C)
; 2. OR them → output result
; 3. AND them → output result
; 4. XOR them → output result
; 5. Rotate the XOR result right by 2 bits → output result
; 6. Halt
; ------------------------------------------------

200F ;Load immediate value 0F (=15) into register 0
213C ;Load immediate value 3C (=60) into register 1

7201 ;OR the values in register 0 and 1 , put result in register 2 (result=3F)
32FF ;Store register 2 into memory[FF] (screen output, prints 3F)

8301 ;AND the values in register 0 and 1 , put result in register 3
(result=0C)
33FF ;Store register 3 into memory[FF] (screen output, prints 0C)

9401 ;XOR the values in register 0 and 1 , put result in register 4
(result=33)
34FF ;Store register 4 into memory[FF] (screen output, prints 33)

A402 ;Rotate register 4 right by 2 bits (33 becomes CC)
34FF ;Store register 2 into memory[FF] (screen output, prints CC)


C000;HALT
```

SECTION 06

GITHUB REPOSITORY

06

REPO LINK *https://github.com/AbdoDawam/FCAI-OOP-Assignment2*

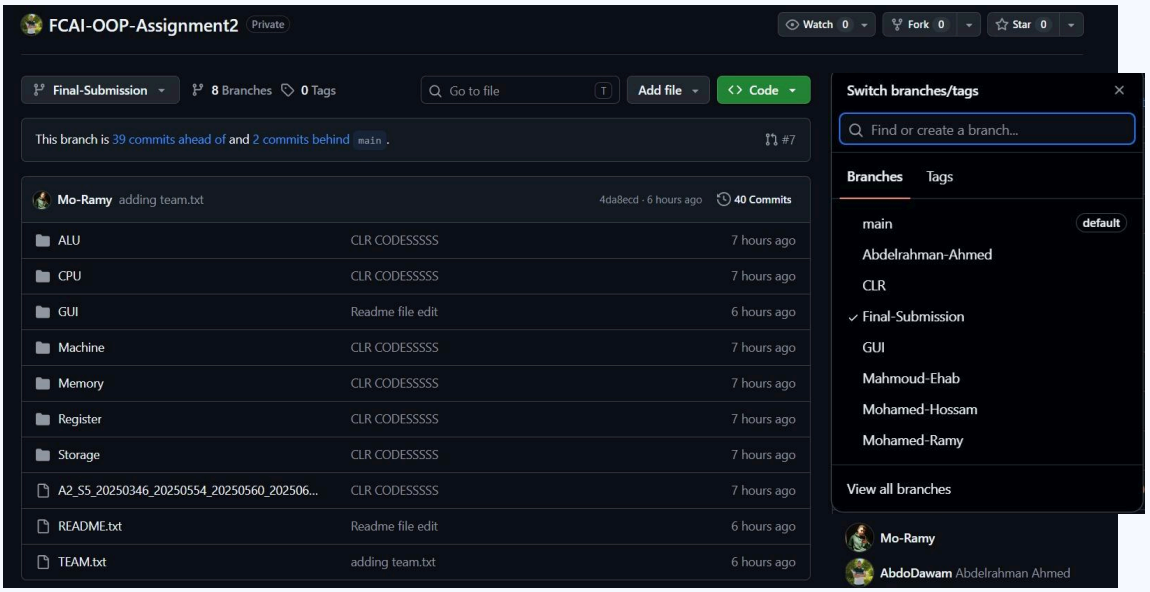


FIG. 06.1 Final sub & Branches

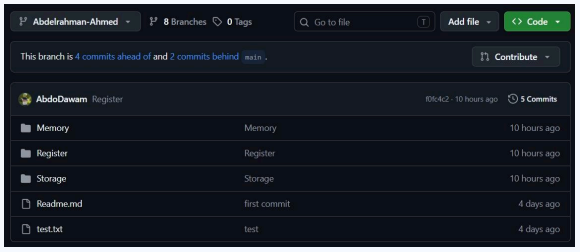


FIG. 06.2 Abdelrahman Ahmed Branch

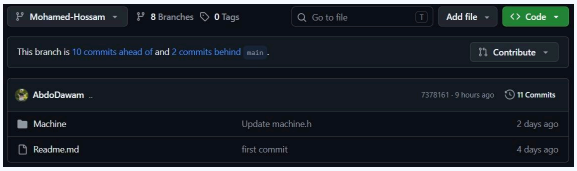


FIG. 06.3 Mohamed Hossam Branch

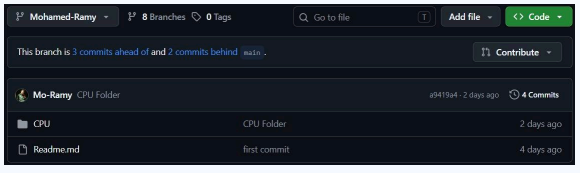


FIG. 06.4 Mohamed Ramy Branch

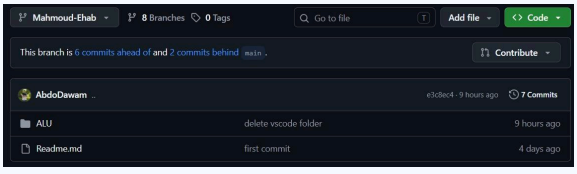


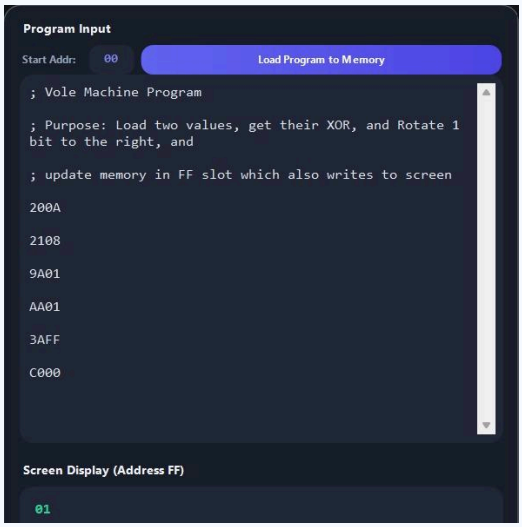
FIG. 06.5 Mahmoud Ehab Branch

SECTION 07

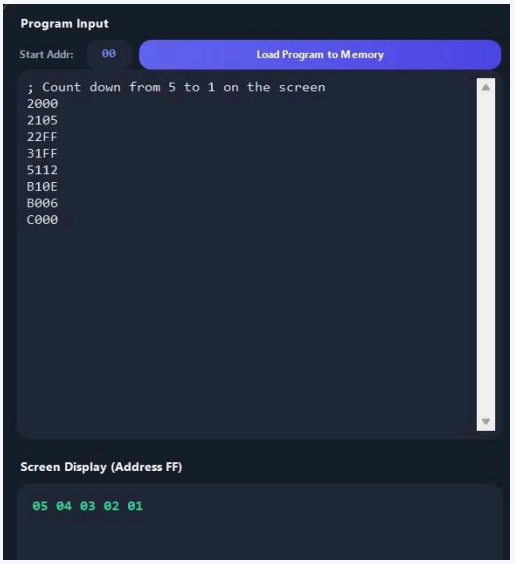
SUCCESSFUL EXECUTIONS

07

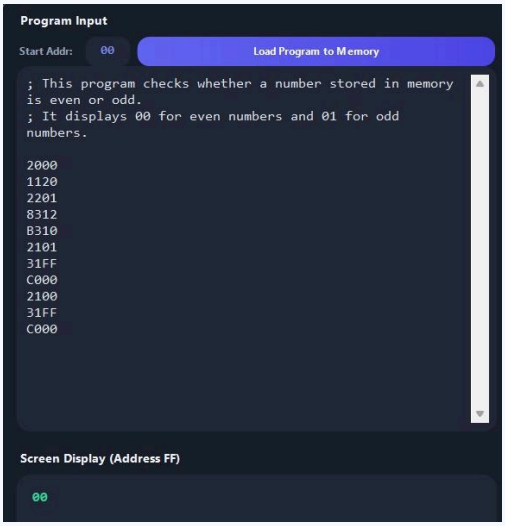
One screenshot for each of the ten programs, showing a successful run.



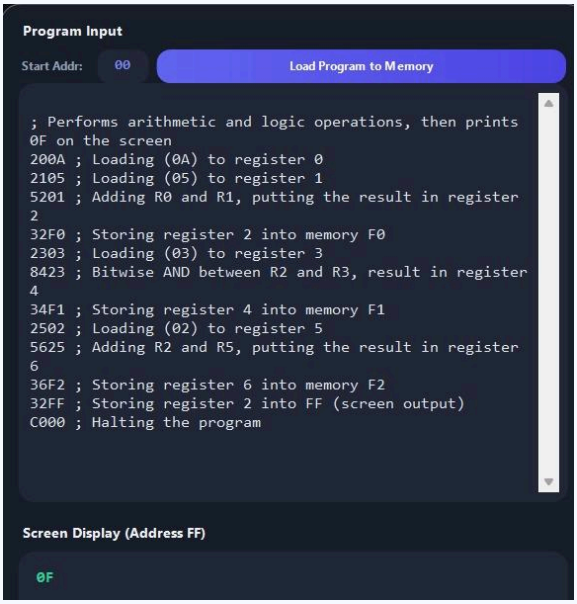
PROG. 01



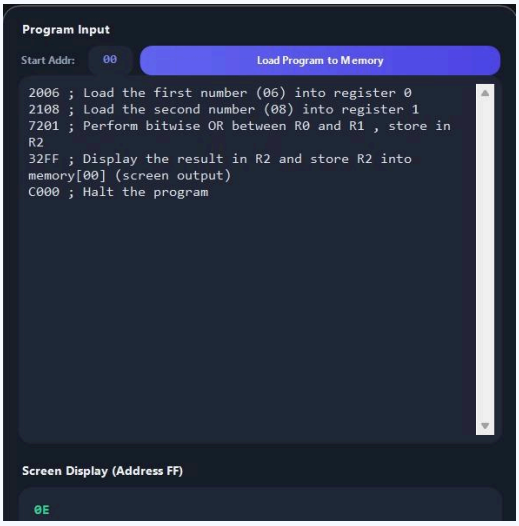
PROG. 02



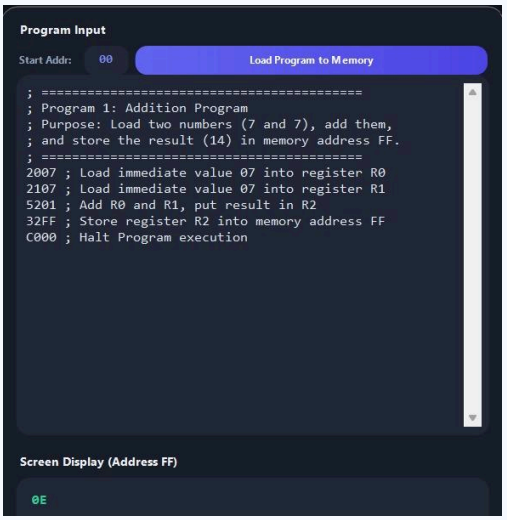
PROG. 03



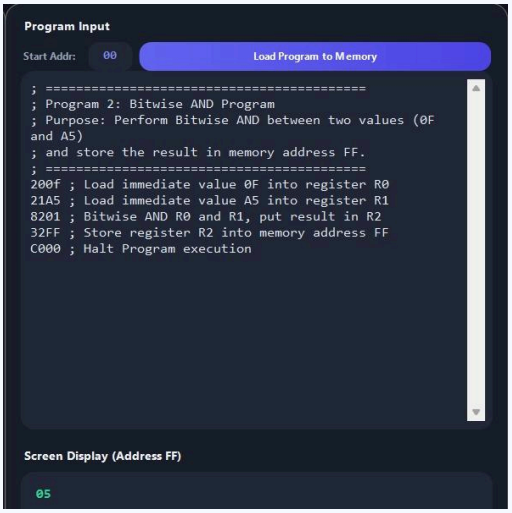
PROG. 04



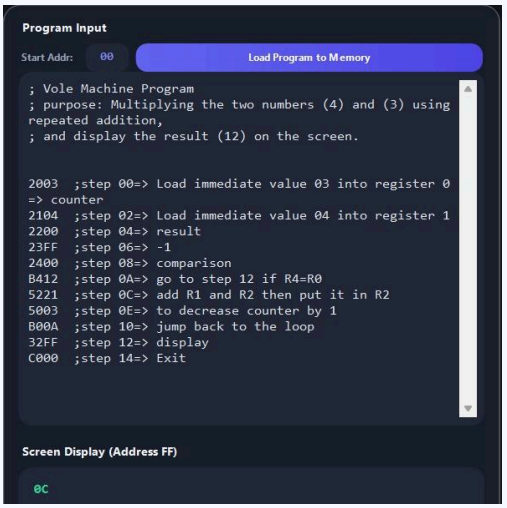
PROG. 05



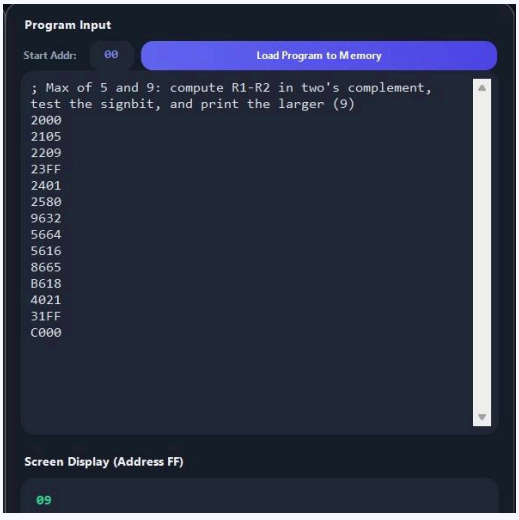
PROG. 06



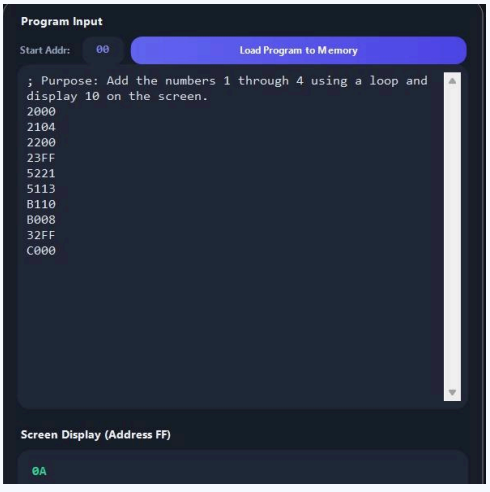
PROG. 07



PROG. 08



PROG. 09

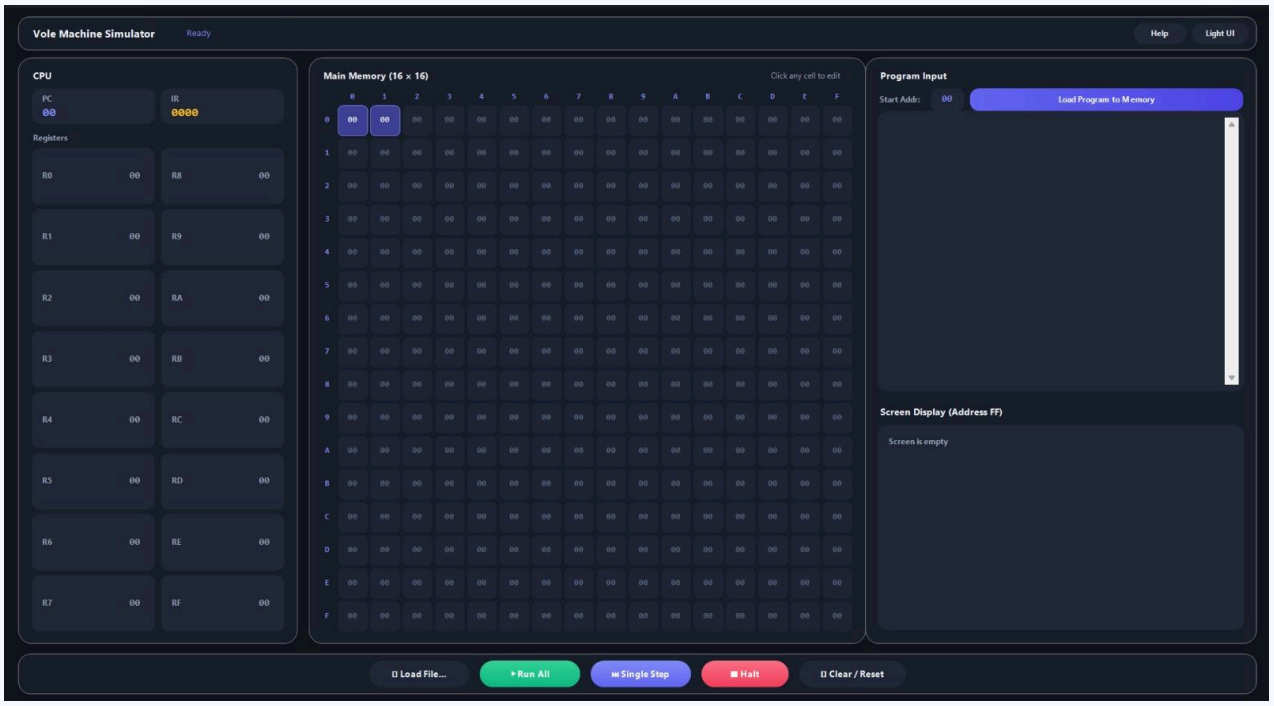


PROG. 10

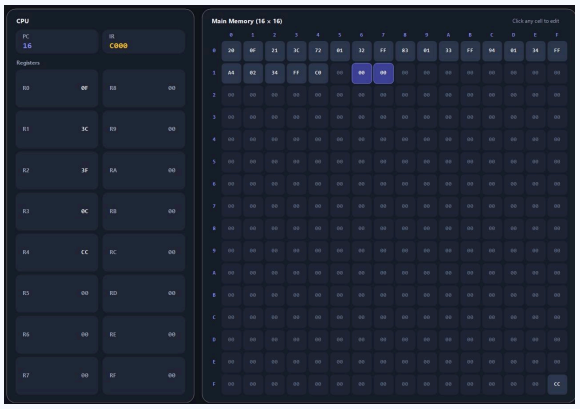
SECTION 08

GUI SCREENSHOTS

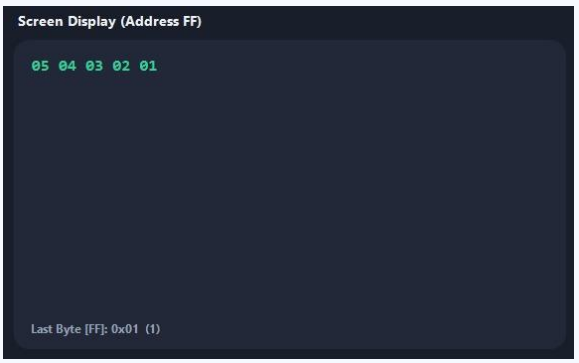
08



GUI 01 The Whole Program



GUI 02 Registers & Memory



GUI 03 The Screen

SECTION 09

VIDEO LINKS

09

AI DESIGN PROCESS	https://drive.google.com/file/d/1GWYQtquJW_TSAd0xNAMjO-2qyWC_rN8o/view?usp=drivesdk
THE GUI	https://drive.google.com/file/d/1quAO92AO2g69pcsW3gSTsNNYNgcOtdPx/view?usp=drivesdk